ARCHITECTURE AND EXECUTION PLAN

Native launcher in Rust + GPUI

Staged decisions for building a command palette of maximum visual quality on Windows and, later, Linux / Wayland.

Version 0.3 — working draft

Date 26 August 2026

Primary goal Personal use, product quality, open source

Platforms Windows 11 (now) → Arch Linux + Hyprland (later)

Working method Development assisted by Claude Code

Contents

1 Vision and non-goals

2 Ecosystem references and licensing

3 Architecture decisions (AD-01 … AD-16)

4 Project invariants

5 Performance budgets

6 Staged plan — Phases F0 to F5

7 Platform recipe: Windows

8 Platform recipe: Linux / Wayland

9 Extensibility: themes, tiers and protocol

10 Risk register

11 Working method with Claude Code

1 Vision and non-goals

What it is

A native desktop command palette written entirely in Rust, with its interface built on GPUI. The declared priority is **quality**: imperceptible latency, a near-zero idle footprint, and an aesthetic on par with Raycast. It is developed first on Windows 11 and designed from day one to be ported to Arch Linux with Hyprland.

What it is not

- **Not a commercial product.** There is no market pressure, no launch date, nobody to beat. If it grows, fine; that is not the design criterion.
- **Not aiming for compatibility with the Raycast ecosystem.** Neither its store nor its API. See AD-10.
- **Not attempting to support every Linux desktop environment.** The target is Hyprland and compositors supporting `wlr-layer-shell`. GNOME is explicitly out of scope.
- **Not a project for accumulating features.** Every UI primitive added must be rendered well on two platforms, forever.

The success criterion

That it becomes the application the author opens fifty times a day for a year without it ever once feeling slow, ugly or fragile. Everything else is secondary.

> NOTE ON VISUAL PARITY
>
> GPUI draws every pixel itself: it does not inherit system theming the way Qt does, nor depend on a WebView engine. The consequence is that visual parity between Windows and Hyprland will be *greater* than what existing alternatives achieve. Only two elements vary by platform: background blur (Acrylic via DWM on Windows; compositor rules on Hyprland, both available) and text rasterisation, mitigated by bundling the typeface rather than using the system one.

2 Ecosystem references and licensing

Three projects define the design space. None is copied; all three are studied.

Project	Stack	What it contributes here
Vicinae GPL-3.0	C++23, Qt / QML, Node.js + React extensions	The closest architectural reference. A native host rendering UI trees emitted by external extension processes. Real-world validation that the model works at scale.
Asyar GPL-3.0 (core) MIT (SDK)	Tauri v2, Rust backend, Svelte 5 frontend, iframe extensions	Permission model declared in a manifest and enforced across two layers. Clipboard privacy layers. Wayland activation pattern. A culture of published benchmarks.
Raycast Proprietary	Native macOS, React extensions	The benchmark for visual and interaction quality. An aesthetic reference, not a technical one.

Licensing constraint — a decision required before writing code

The cores of Vicinae and Asyar are GPL-3.0. This has concrete consequences:

- **Reading to understand** architecture, protocol shape, manifest structure: unrestricted, and the highest-value activity available.
- **Copying code**: forces this project to be GPL-3.0 too. That may be acceptable, but it must be a conscious prior decision, not a later discovery.
- Asyar's SDK and extension tooling are MIT and are reusable with attribution.

Measured baseline (July 2026 reference, Apple M4 Max)

Metric	Asyar (Tauri)	Raycast	Target here
Hotkey → window visible (p50)	12.0 ms	21.7 ms	< 40 ms
Cold start to usable	572 ms	888 ms	< 400 ms
Idle RAM	435.6 MB	272.6 MB	**< 150 MB**
Idle CPU (30 s)	3.20 %	0.04 %	**< 0.2 %**
On-disk size	64 MB	209 MB	< 40 MB

> HOW TO READ THIS TABLE
>
> Open latency **is not the differentiator**: Tauri already reaches 12 ms and there is no perceptible gain left there. The real differentiator is the idle column. A launcher spends 99.9 % of its time hidden; 435 MB and 3.2 % CPU permanently are the price of the WebView and translate into battery burned all day. That — not open speed — is what justifies going native.

3 Architecture decisions

Each decision states the choice and the reason. They are the project's contract: revising them is legitimate, ignoring them silently is not.

AD-01

Rust throughout, with GPUI as the interface layer

GPUI renders on the GPU and draws all of its own controls, guaranteeing exact visual parity across platforms. The accepted cost is that it is pre-1.0, with breaking changes between versions and sparse documentation: the practical reference is Zed's source.

AD-02

Resident daemon plus a client that toggles visibility over IPC

Solves two problems at once. First, startup: the window is created once and merely hidden, eliminating GPU initialisation cost on every invocation. Second, Wayland: since no standard global shortcut API exists, the accepted pattern is for the compositor to execute the binary and for a second execution to toggle the existing instance. This is the same design Asyar uses in production.

AD-03

Four layers: core, platform, ui, app

`core` holds the index, matching, providers and actions, with no knowledge of the operating system or of GPUI. `ui` depends on GPUI. `platform` defines the system boundary. If GPUI turns out to be unworkable at some point, the presentation layer is replaced without losing the project.

AD-04

A single `trait Platform` as the operating-system boundary

A deliberately small surface: global shortcut registration, show and hide window, save and restore previous focus, choose monitor, enumerate applications, extract icons, clipboard and autostart. Any need to cross that boundary with anything else signals coupling and must be fixed, not accommodated by widening it.

AD-05

The theme is a data file, not code, and reloads live

Aesthetics are iterated hundreds of times. Recompiling on every iteration kills the design process. A token file (colours, spacing, radii, typography) re-read on change is the afternoon of work with the highest return in the entire project.

AD-06

The type contract is defined once and is transport-independent

The UI tree and event types are declared as Rust structs with `serde`, and that is the contract. From it are derived both the WIT interface for a WebAssembly guest and the JSON schema for any out-of-process runtime. Declaring the API directly in WIT would couple the system to WebAssembly forever; declaring it only as JSON would prevent strong typing in a Rust guest. The contract is the durable artefact; the transport is a reversible decision.

AD-07

A closed, small UI vocabulary

Extensions never draw pixels: they emit a declarative tree over a fixed set of primitives (List, Detail, Form, Grid, ActionPanel) that the host renders natively. The intended consequence is that no extension can break the aesthetic of the whole. Consistency is obtained by construction, not by third-party discipline.

AD-08

Static manifest indexed at install time

When the user types in root search, zero extension processes are running. Commands appear because they are in the local index, not because anyone reported them. The child process is launched only when a command is activated.

AD-09

Every extension capability goes through the host RPC

Network, filesystem, execution, clipboard and notifications pass through the host even where the subprocess could technically perform them directly. There will be no real sandbox at first and that is acceptable for personal use; what matters is that the boundary is drawn, so that introducing a sandbox later means changing the runtime rather than redesigning the system. Asyar's permission taxonomy serves as the starting point.

AD-10

No compatibility with Raycast's API or store

It would mean implementing their React reconciler, their full API surface, npm resolution, and inheriting trademark and terms-of-service risk that has already affected other projects. What is adopted is the *shape* of their API — same component names, same action-panel structure — so that porting an extension is mechanical without promising compatibility.

AD-11

Licence chosen before the first commit

Deciding later means relicensing. The choice also conditions which third-party code may be read with intent to reuse. See section 2.

AD-12

Themes are pure data, with no code and no runtime

This is Zed's model, where a theme is a colour definition file and nothing more. A community theme installs by dropping a file into a folder: no risk, no sandbox, no process. Tokens are semantic (`surface.background` , `text.muted` , `accent`) and never name components, because a token that names a component turns every new component into a break for every existing theme.

AD-13

Themes control colour and typography, never geometry

No spacing, no radii, no row heights, no icon sizes. If a third-party theme can alter geometry, the project's carefully calibrated layout dies the first day someone publishes one. This is the decision that specifically protects what motivates the project.

AD-14

Extensibility in four tiers, not one mechanism

Each tier has a radically different effort-to-value ratio and deserves a separate decision: themes as data (T0), script commands as executables emitting JSON (T1), view extensions compiled to WebAssembly (T2) and, optionally and later, an alternative JavaScript runtime over the same contract (T3). See section 9.

AD-15

Keyboard handling operates at two levels with a sharp boundary

Outside the window — global gestures, launcher invocation, press-and-hold gesture — is bespoke platform code behind `trait Platform` . **Inside the window** is GPUI's action and keymap system, with no in-house substitute. GPUI resolves keystrokes against the focus tree, which requires the window to already hold focus; that is why it does not, and will not, cover global invocation: Zed never needs to be summoned from outside. Neither layer intrudes on the other.

AD-16

The press-and-hold gesture is not a shortcut and is not implemented as one

System accelerator APIs require modifiers plus a non-modifier key, and report only the press. The gesture required by the shortcut overlay is a lone modifier, held, with measured duration and a release event: it is a gesture over keyboard state, not an accelerator. No global-shortcut library covers it because it does not fit their abstraction, and that is the criterion by which bespoke code is written here instead of taking a dependency.

4 Project invariants

Rules that are not negotiated case by case. They are written to be copied verbatim into the repository's `CLAUDE.md`.

#	Invariant
I-1	`core` contains no `#[cfg(windows)]` or `#[cfg(unix)]` directive, and no direct operating-system call. Everything goes through `trait Platform`.
I-2	`platform-linux` always compiles, even if its methods are `todo!()`. CI runs `cargo check --target x86_64-unknown-linux-gnu` on every commit.
I-3	Root search never waits on an extension process. No exceptions and no "short" timeouts.
I-4	The window is created once when the daemon starts and from then on is only shown or hidden. Never destroyed.
I-5	No colour, spacing, radius or type size appears literally in UI code. Everything comes from the token file.
I-6	The `gpui` version is pinned exactly in `Cargo.toml`. Never `*`, never a range. Updating is a deliberate commit with its own verification.
I-7	Every capability an extension needs is requested over host RPC. The subprocess does not reach resources directly even where the system would allow it.
I-8	The performance budgets in section 5 are measured in CI. A regression breaks the build.
I-9	Changing the signature of `trait Platform` requires updating *both* implementations in the same commit.
I-10	Fixed row height and reserved space for icons and accessories. Zero layout shift when results change.
I-11	A theme may define only colour and type family. No geometric property is themeable.
I-12	Every missing theme token resolves from the base theme. A theme written today must still load a year from now.
I-13	The UI tree and event types exist in exactly one place. No parallel definitions in WIT, in JSON Schema or in the SDK.
I-14	In-app shortcuts are declared as GPUI actions and bound from a keymap file. No hand-written key comparisons in UI code.
I-15	No system input callback allocates, logs, takes a lock or touches UI state. It pushes the event to a channel and returns.

5 Performance budgets

Numbers, not adjectives. Measured from phase F1 onward by a reproducible script that runs in CI and publishes results in the README, following Asyar's practice.

Budget	Threshold	How it is measured
Global gesture → first frame on screen	**p50 < 40 ms** **p95 < 80 ms**	Synthetic keypress until the window is visible. Median of 15 repetitions on an idle machine.
Per-frame cost while typing	**p95 < 8 ms**	Internal instrumentation of the render loop with the full list loaded.
Idle resident memory	**< 150 MB**	Sum across the process group with the window hidden and no extensions active.
Idle CPU	**< 0.2 %**	Average over 30 seconds with the window hidden.
Cold start to usable	**< 400 ms**	Daemon launch until the first search responds.
Installed size	**< 40 MB**	Binary plus bundled assets.

> **WHY THIS LANDS IN F1 AND NOT AT THE END**
>
> In a project where perceived quality is the declared goal, having measurement from the start avoids the usual fate: discovering six months in that something introduced slowness and being unable to identify which commit did it.

6 Staged plan

The order is deliberately inverted relative to instinct. Risks that could invalidate the whole project are attacked first; aesthetics, which is the main motivation, arrives once there is a base to support it.

PHASE F0

Platform spike on Windows

Estimate: 1 – 3 weeks · Risk: maximum

A single window and nothing else. This is the phase that decides whether the aesthetic plan is viable as described.

DELIVERABLES

- Undecorated, transparent GPUI window with rounded corners and its own shadow.
- Acrylic backdrop active via `DWMWA_SYSTEMBACKDROP_TYPE` set to `DWMSBT_TRANSIENTWINDOW`.
- Appears centred on the monitor holding the cursor, with per-monitor DPI awareness.
- Global gesture registered; focus taken reliably and returned to the previous window on close.

ACCEPTANCE CRITERIA

- The window shows real blur of the content behind it.
- It does not appear in Alt+Tab or the taskbar.
- After closing, focus returns to exactly the previous application across 20 consecutive attempts.
- Correct behaviour on dual-monitor setups with differing scale factors.

PRINCIPAL RISK

The system backdrop is visible only if the client area is genuinely transparent: no opaque clear, and correct premultiplied alpha in the renderer. This phase will likely end in a patch to `gpui_platform`. If it proves unworkable, plan B is a solid dark surface with a subtle gradient — visually very defensible — and the rest of this document is unchanged.

PHASE F1

The minimum usable launcher

Estimate: 1 – 2 months · Risk: medium

Goal: the author installs it as the default launcher and uninstalls the previous one. Still ugly, but real.

DELIVERABLES

- Resident daemon plus a thin client binary that toggles visibility over a local socket.
- Application index: Start Menu shortcuts plus packaged apps via WinRT.
- High-resolution icon extraction via `IShellItemImageFactory`.
- Fuzzy matching with `nucleo`, ranked by frequency and recency of use.
- Benchmark harness running in CI against the thresholds in section 5.

ACCEPTANCE CRITERIA

- All performance budgets are met and published.
- The index covers 100 % of installed applications, packaged ones included.
- No icon appears pixelated at a 32 px row size.

PRINCIPAL RISK

Icons. This is where these projects look cheap and where the real effort is always underestimated. Budget explicit time for extraction, scaling and caching.

PHASE F2

Design system and aesthetics

Estimate: continuous, never finished · Risk: low

The phase that motivated the project. It arrives now because it needs a base that will not shift underneath it.

DELIVERABLES

- Token file with live reload: colour, spacing, radius, typography, elevation.
- **Tier T0 — theme system:** versioned theme format with `schema_version`, semantic tokens, default resolution from the base theme, and loading from a watched folder. At least two in-house themes, one light and one dark, to validate the schema before opening it to third parties.
- Typeface bundled into the binary (Inter or Geist), never the system one.
- A single icon set optically aligned on a 16 and 20 px grid.
- Action panel and navigation fully operable from the keyboard.
- Appearance animation: fade plus a 0.98-to-1.0 scale over roughly 120 ms. Nothing more.

ACCEPTANCE CRITERIA

- Changing a token in the file is reflected on screen without recompiling.
- Zero layout shift when results change (invariant I-10).
- Scrolling jumps directly to the selection; there is no smooth scrolling.
- Every action reachable without a mouse.

PRINCIPAL RISK

The temptation to animate everything. Restraint is what produces the sense of quality; excess motion is the most common error and the hardest to walk back later.

PHASE F3

Internal providers

Estimate: 1 – 2 months · Risk: medium

Everything that makes the launcher useful before an extension system exists. Each one behind a `trait Provider` in `core`.

SUGGESTED DELIVERABLES, IN ORDER OF VALUE

- Inline calculator in the search bar.
- Clipboard history persisted in SQLite.
- Snippets and text expansion.
- Window management with layout presets.
- File search with an in-house in-memory index.

TIER T1 — SCRIPT COMMANDS

Any executable that prints JSON, with metadata declared in comment headers. It costs days and covers a large fraction of real use cases without touching the extension system. It is the best effort-to-value ratio in the entire project and must not be deferred to F4.

CONTEXT-AWARE SHORTCUT OVERLAY

Holding a configurable key shows a shortcut sheet annotated by the user; releasing it hides the sheet. The difference from the equivalent PowerToys utility is twofold: the shortcuts are *the user's own*, not the operating system's fixed set, and the sheet shown depends on the foreground application. The context data is already captured for focus restoration, so the incremental cost is minimal.

- **Shortcut sheets are tier T0 data files**, exactly like themes: same distribution model, no new runtime, contributable by the community.
- The application's own shortcuts are not written by hand: they are generated by enumerating GPUI's global action registry.
- Mandatory fallback chain: match by executable, then pattern on window title, then global sheet. The overlay never appears empty.
- It is a **new platform surface**, not a UI feature: its own window, click-through, non-activating, no focus stealing, and its own layer-shell surface on Wayland. It widens `trait Platform`.

OVERLAY ANIMATION RULES

- Pre-created hidden window, like the launcher's. That is two warm windows in memory, not one.
- The animation starts at zero opacity: if the first frame arrives late it is invisible, and the hitch hides itself.
- Animate opacity and transform only. Never widths, heights or positions, which force per-frame layout recalculation.
- Entry around 150 ms with ease-out; exit faster, around 90 ms. Equal durations feel heavy.
- **The animation must be interruptible and reversible from its current value.** On a quick release, restarting from opacity one produces a visible jump. This is the most common failure and the reason many overlays feel cheap.
- The activation threshold sits around 300–500 ms. Three seconds reads as a broken feature.
- The overlay is passive: it never consumes the following keystroke. If it interferes with the shortcuts it documents, it stops being useful.
- On Hyprland, decide whether the compositor or the application animates. Both at once looks wrong.

PRIVACY LAYER — ADOPTED FROM ASYAR

Cheap to implement in Rust and with a high return in perceived quality: a denylist of password managers, respect for the "do not include in history" clipboard flags on Windows, and regex redaction of known secret formats *before* writing to disk.

ACCEPTANCE CRITERIA

- No provider introduces UI-thread work that exceeds the per-frame budget.
- Clipboard history never stores content marked sensitive by the system or by the denylist.

PRINCIPAL RISK

The clipboard is the first component with genuine persistent-state complexity. It requires a system event listener, schema migrations and a retention policy. Treat it as a subproject, not as one more feature.

PHASE F4

View extensions (tiers T2 and T3)

Estimate: open-ended · Risk: high

This is where the bulk of long-term work lives. By this point T0 and T1 have been in production since earlier phases and already cover much of the real extensibility. Build order matters more here than anywhere else.

MANDATORY SEQUENCE

1. Hand-write the JSON of a `List` tree with ten items and an action panel. Render it in GPUI from a static file. No protocol, no processes.
2. Wrap that rendering behind a `render_node(node) -> AnyElement` function, with list virtualisation.
3. Only then move the tree across a channel and introduce events.
4. Write the **second** extension before stabilising anything in the protocol.

ACCEPTANCE CRITERIA

- An isolated instance per extension, created lazily and destroyed on idle.
- An extension failure shows an error in its view and does not affect the launcher.
- Root search still responds within budget with ten extensions installed and none active.
- The type contract has a single definition from which per-transport representations are generated (I-13).
- Memory limits and epoch interruption for the WebAssembly guest are active and tested with a deliberate infinite loop.

PRINCIPAL RISKS

Coupling to the runtime. The protocol must survive a move from a generic subprocess to embedded JS or to WASM.

The first extension lies. It fits perfectly because the protocol was designed while looking at it. The second reveals the mistakes.

Features that break AD-08. Live reactive subtitles are attractive but require live processes at the root. If implemented, they must be opt-in and limited to already-warm extensions; the root never waits.

PHASE F5

Port to Linux and Wayland

Estimate: 3 – 6 weeks if I-1 and I-2 were respected · Risk: medium

Triggered by the migration to Arch. If the invariants held, this is implementing an already-defined trait, not rewriting.

DELIVERABLES

- `wlr-layer-shell` surface on the `LAYER_OVERLAY` layer with exclusive keyboard interactivity.
- Activation delegated to the compositor: one `bind` line in the Hyprland configuration that runs the client.
- Application index from `.desktop` files per the XDG specification, with its icon themes.
- Blur via compositor rules.

ACCEPTANCE CRITERIA

- The same performance budgets are met on Linux.
- Side-by-side visual comparison with Windows: indistinguishable apart from background blur.

PRINCIPAL RISK

GPUI does not implement `wlr-layer-shell`. A patch or fork of the Wayland backend will have to be maintained. Accepting this now turns a surprise into a planned task.

7 Platform recipe: Windows

Window styles

- `WS_EX_TOOLWINDOW` — excludes the window from Alt+Tab and the taskbar.
- `WS_EX_TOPMOST` — keeps the window above the rest.
- `WS_EX_NOACTIVATE` — only if the decision is not to steal focus on appearance.
- No system decoration; the border and shadow are drawn by the application.

Composition attributes (Windows 11 22H2 or later)

Applied with `DwmSetWindowAttribute`:

Attribute	Value and purpose
`DWMWA_SYSTEMBACKDROP_TYPE`	`DWMSBT_TRANSIENTWINDOW` — the Acrylic designed specifically for flyouts and launchers. Do not use Mica, which is intended for persistent application windows.
`DWMWA_WINDOW_CORNER_PREFERENCE`	`DWMWCP_ROUND` — system rounded corners.
`DWMWA_USE_IMMERSIVE_DARK_MODE`	Dark mode consistent with the system.
`DWMWA_BORDER_COLOR`	One-pixel border separating the window from the background.

> **INDISPENSABLE CONDITION**
>
> The system backdrop becomes visible only if the client area is genuinely transparent. This requires a transparent window in GPUI, the absence of an opaque clear, and correct premultiplied alpha. Verifying this is the central objective of F0.

Focus handling

- Capture `GetForegroundWindow()` **before** showing, so it can be restored on close.
- To take focus reliably, attach thread input with `AttachThreadInput` against the foreground window's thread. `SetForegroundWindow` on its own fails intermittently.

Positioning

- Declare per-monitor DPI awareness version 2 in the application manifest.
- Choose the monitor with `MonitorFromPoint` on the cursor position. The launcher must appear where the user's attention is, not on the primary monitor.

Application index and icons

- Start Menu shortcuts in the system and user paths.
- Packaged applications via WinRT's `PackageManager` through the `windows` crate.
- **Icons via `IShellItemImageFactory`**, not the legacy extraction functions: those return low-resolution bitmaps that ruin the entire aesthetic.

Global keyboard input

There are three mechanisms and the choice deserves care, because the press-and-hold gesture described in F3 does not fit the accelerator model (AD-16): `RegisterHotKey` requires a non-modifier key and does not report release.

Mechanism	Suppresses	Detects release	Cost
`RegisterHotKey`	Yes	No	None, no hook
`Raw Input (RIDEV_INPUTSINK)`	No	Yes	Low, no timeout risk
`WH_KEYBOARD_LL`	Yes	Yes	Global hook with timeout

Default choice: Raw Input as the single event source, from which both launcher invocation and the hold gesture are derived. One source of truth is preferable to two event loops coexisting. The hidden message window plus `WM_INPUT` handling runs to roughly 150–250 lines.

When the low-level hook is needed: only if the gesture key is Win, whose release opens the Start menu and must be suppressed. Choosing any other key avoids the hook entirely. The key must be configurable.

> **IF WH_KEYBOARD_LL IS ULTIMATELY USED**
>
> The callback must return within microseconds. Windows silently unhooks it once `LowLevelHooksTimeout` — 300 ms by default — is exceeded, and reports nothing: events simply stop arriving. No allocation, logging, lock acquisition or UI state access inside the callback. The event is pushed to a lock-free channel and the callback returns immediately. Global hooks also draw suspicion from some antivirus and anti-cheat systems.

On existing libraries

The `global-hotkey` crate from the Tauri ecosystem is well maintained and covers Windows, macOS and X11, but does not solve this case for two reasons: it lacks Wayland support — proposals have been open since 2025 without landing — and it exposes press only, not release. What is worth reusing is its key vocabulary via the `keyboard-types` crate, which supplies portable code and modifier enums without dragging in an event model or platform layer. Translating between physical scancodes and virtual key codes is the tedious, easy-to-get-wrong part; the rest is not.

8 Platform recipe: Linux and Wayland

Window presentation

Wayland offers no standard way for an application to place itself above others or to position itself on screen. The solution is the `wlr-layer-shell` protocol, supported by Hyprland, Sway and KWin, with the surface on the `LAYER_OVERLAY` layer and exclusive keyboard interactivity.

> **DECLARED SCOPE**
>
> GNOME and Mutter do not support `wlr-layer-shell`. They are explicitly out of scope. Stating this in the README prevents expectations that cannot be met.

Activation

There is no global shortcut API on Wayland. The established pattern, used by Asyar and by the rest of the ecosystem's launchers, is to delegate the shortcut to the compositor:

```
# ~/.config/hypr/hyprland.conf
bind = ALT, SPACE, exec, mylauncher
```

Running the binary when a resident instance already exists toggles its visibility through the local socket. This is exactly the AD-02 architecture, which is why it must exist from F1 even though Windows would not strictly require it.

PRESS-AND-HOLD GESTURES

Hyprland provides separate press and release binds, which allows the overlay gesture to be implemented with no input-capture code and no special permissions:

```
bind  = , SUPER_L, exec, mylauncher overlay-press
bindr = , SUPER_L, exec, mylauncher overlay-release
```

Two thin-client invocations, two socket messages, and the daemon computes the duration. For compositors without a release bind, plan B is reading `/dev/input` with the `evdev` crate: it works on any compositor because it reads kernel events before the compositor processes them, and it provides both press and release, at the cost of requiring membership of the `input` group and exposing every keystroke on the system. It must be an explicit option, never the default path.

The XDG GlobalShortcuts portal is the standard and correct long-term route, but it is press-oriented and does not cover the gesture today.

Foreground application detection

Hyprland's event socket exposes the active window's class and title directly, with no additional calls. It is simpler than on Windows, where it requires chaining `GetForegroundWindow`, `GetWindowThreadProcessId` and `QueryFullProcessImageNameW`.

Application index

- `.desktop` files per the XDG desktop entry specification.
- Icon resolution following the icon theme specification, with size fallbacks.
- Directory watching with `inotify` for incremental reindexing.

Aesthetics

Blur is obtained through compositor rules, not from the application. On Hyprland it is declared for the relevant window class. Rounded corners and shadow are drawn by the application, since on Wayland decoration is the client's responsibility.

9 Extensibility: themes, tiers and protocol

The lesson from Zed, and its limit

Zed's extensions are Rust crates compiled to `wasm32-wasip2` and run in a `wasmtime` sandbox, with a manifest declaring capabilities. It is an excellent design. But it is worth reading precisely what it solves: Zed's own documentation states that its extensions can contribute language support, debuggers, MCP servers, themes and icon themes, and that allowing UI customisation remains a future plan. An open proposal for a visual API exists and is still unresolved.

> THE DISTINCTION THAT DETERMINES THE WHOLE DESIGN
>
> Zed's extensions are **capabilities**: they register a binary, return metadata, configure a server. They do not draw. A launcher's extensions are roughly 90 % interface. Zed has had WebAssembly for years and still has not opened guest-side UI rendering, precisely because that is the hard problem. Copying their model wholesale would be copying the answer to a different question. What is adopted in full is their theme model.

The four tiers

Tier	What it is	Cost to build	Friction for authors	Phase
`T0`	Themes: data files	Very low	None	F2
`T1`	Script commands: any executable emitting JSON	Very low	None	F3
`T2`	View extensions: Rust compiled to WebAssembly	High	High	F4
`T3`	Alternative JavaScript runtime over the same contract	Medium	Low	Optional

T0 — Theme format

Pure data, no code, no runtime. Four design rules:

- **Semantic tokens.** `surface.background`, `text.muted`, `accent`, `selection.background`. Never component names such as `list_item_hover`.
- **Mandatory default resolution.** Every missing token falls back to the base theme, so widening the palette does not invalidate existing themes.
- **Versioned schema** from the first file, so migration replaces breakage.
- **No geometry.** Colour and type family exclusively (AD-13, I-11).

```
{
  "schema_version": 1,
  "name": "Nocturne",
  "appearance": "dark",
  "colors": {
    "surface.background":   "#0E0E12",
    "surface.elevated":     "#17171B",
    "border.subtle":        "#26262E",
    "text.primary":         "#EDEDF2",
    "text.muted":           "#8A8A96",
    "accent":               "#7C6BD6",
    "selection.background": "#221F33"
  }
}
```

The same format and the same distribution mechanism serve the shortcut sheets used by the context-aware overlay in F3.

T1 — Script commands

An executable in any language that prints results as JSON, with metadata declared in comment headers in the style of Raycast and Vicinae. No bidirectional protocol, no state, no views. Days of work covering a very wide fraction of real cases.

T2 and T3 — The protocol with views

The remainder of this section describes tier T2. Tier T3 reuses the very same type contract (AD-06) over a different transport, which is why adding it later means adding a backend rather than redesigning the system.

NOTE ON THE EXECUTION MODEL

Unlike a generic subprocess, a WebAssembly guest can run **inside the host process** with real isolation, as Zed does: `wasmtime` allows bounding guest memory and using epoch interruption to abort infinite loops. Isolation is obtained without paying for a process per extension, which directly favours the budgets in section 5.

The three pieces

1. **Static manifest.** Declares commands, permissions and assets. Indexed at install time; nothing needs to be executed.
2. **Events from host to extension.** A minimal surface: search text changed, selection changed, action invoked, form submitted, view closed.
3. **UI trees back.** The extension responds with a complete tree. Nodes carry stable identifiers to preserve focus and scroll. Do not build incremental diffing until a measurement justifies it.

Manifest sketch

```
{
  "id": "github",
  "name": "GitHub",
  "commands": [
    {
      "id": "search-repos",
      "title": "Search repositories",
      "mode": "view",
      "icon": "assets/gh.svg"
    }
  ],
  "permissions": ["network", "clipboard:write", "shell:open-url"]
}
```

UI tree sketch

```
{
  "type": "List",
  "isLoading": false,
  "items": [
    {
      "id": "repo-1",
      "title": "vicinaehq/vicinae",
      "subtitle": "C++23 · GPL-3.0",
      "accessories": [{ "text": "8.9k" }],
      "actions": [
        { "id": "open", "title": "Open in browser", "primary": true },
        { "id": "copy", "title": "Copy URL" }
      ]
    }
  ]
}
```

Permission taxonomy

A starting point adopted from Asyar, sufficient to cover the real case space without bloating:

Permission	Grants
`clipboard:read / write`	Access to the system clipboard
`fs:read / write`	Reading or writing files
`network`	Outbound HTTP requests
`shell:execute`	Command execution
`shell:open-url`	Opening URLs in the browser
`notifications:send`	System notifications
`store:read / write`	Extension-owned data persistence

Execution model

- One isolated instance per extension, not per command.
- Created on first use; kept warm for a few minutes; destroyed on idle.
- A failure becomes an error message inside the view. The launcher never notices.

Developer tooling — not optional

- **Dev mode with live reload:** a command that watches a directory and restarts the guest on change. Without it, writing extensions is painful and few will be written.
- **Protocol inspector:** a flag that dumps RPC messages to a file. It is the difference between ten minutes and an afternoon when something misbehaves.

10 Risk register

#	Risk	Phase	Mitigation
R-1	Real transparency proves unworkable in GPUI on Windows and the system backdrop does not show	F0	Verified in the first week. Plan B: solid dark surface with a subtle gradient, visually defensible and with no impact on the rest of the plan.
R-2	Breaking changes in GPUI between versions	All	Exactly pinned version (I-6). Updates as deliberate commits with their own verification, never by inertia.
R-3	Insufficient GPUI documentation	All	The practical reference is Zed's UI source. Keep a local checkout to point at during development.
R-4	Windows assumptions leaking into the core, making porting equivalent to rewriting	F1 – F4	Invariants I-1 and I-2, with cross-checking in CI from the first commit. Detected the same day, not a year later.
R-5	GPUI does not implement `wlr-layer-shell`	F5	Accept maintaining a Wayland backend patch as a planned task with its own estimate.
R-6	The extension protocol becomes coupled to its first runtime	F4	Design the contract before choosing a runtime (AD-06). Every capability via host RPC (AD-09).
R-7	Uncontrolled growth of the UI primitive vocabulary	F4	Every new primitive requires explicit justification. Each must be rendered well on two platforms permanently.
R-8	Licence contamination from inadvertent reuse of GPL code	F0	Licence decision before the first commit (AD-11). Study architecture, do not copy implementations.
R-9	Silent performance degradation over months	F1+	Budgets measured in CI (I-8). A regression breaks the build and identifies the responsible commit.
R-10	Loss of motivation from deferring the aesthetic work indefinitely	F2	F0 and F1 are deliberately bounded. Live token reload turns design into an activity measured in minutes, not builds.
R-11	Community themes break as the interface evolves	F2+	Semantic tokens rather than component names, default resolution from the base theme, and a versioned schema (AD-12, I-12). Validate the format with two in-house themes before opening it to third parties.
R-12	WebAssembly compilation friction reduces the number of extensions actually written	F4	Consciously accepted risk: T0 and T1 cover low-effort extensibility and require no compilation. If friction proves excessive, T3 is added without redesign, thanks to AD-06.
R-13	The global keyboard hook is unhooked by timeout or flagged by antivirus	F3	Raw Input as the default mechanism, which has neither timeout nor hook. Reserve `WH_KEYBOARD_LL` strictly for the case where the gesture key requires suppression, under the discipline of I-15.
R-14	Foreground application detection returns the container rather than the real application	F3	Packaged applications report the frame host process and require descending to child windows; the same problem appears in the application index, so it is solved once and reused. Browsers and Electron applications return the container without distinguishing content: declared out of scope.

11 Working method with Claude Code

Repository preparation

- Store this document as `docs/ARCHITECTURE.md` in the repository.
- Create a `CLAUDE.md` at the root containing **only** sections 4 (invariants) and 5 (budgets), plus a link to the full document. A short `CLAUDE.md` is respected; a long one is ignored.
- Configure from the first commit: `cargo fmt --check`, `cargo clippy -- -D warnings` and the Linux target cross-check.

Delegation rules

Type of work	How to approach it
The F0 spike	By hand, not delegated. This is where judgement matters and where the result must be *seen* to be evaluated. Delegating it produces code that compiles and a window that does not look right.
System API integration	Delegable under close supervision. Always verify against official documentation: Win32 signatures and constant values are fertile ground for subtly wrong details.
GPUI code	Point explicitly at Zed's source as a reference in the prompt. A model's prior knowledge of a pre-1.0 library may correspond to an earlier API.
Internal providers, parsing, index, matching	Broad delegation. Pure logic, well specified and verifiable with tests.
UI tree rendering	Delegable once a hand-written working example exists. Not before.
Aesthetic tuning	Not delegable. It requires looking at the screen and deciding. That is what live token reload is for.

Working rhythm

- **One phase, one branch. One acceptance criterion, one pull request.** The criteria in section 6 are written to serve as a definition of done.
- For anything performance-related, **ask for the measurement first and the functionality second**. A benchmark written beforehand is a constraint; written afterwards it is a justification.
- Standing instruction: changing `trait Platform` requires updating both implementations in the same commit (I-9).
- Standing instruction: no literal colour or spacing values in interface code (I-5).

> **THE WARNING THAT WILL WEIGH MOST**
>
> The characteristic risk of assisted development on a project like this is not incorrect code: it is **volume**. It is very easy to accumulate functionality faster than it can be polished, and this project is defined precisely by polish. The invariants and the budgets exist to act as a brake. When they conflict with moving faster, they win.